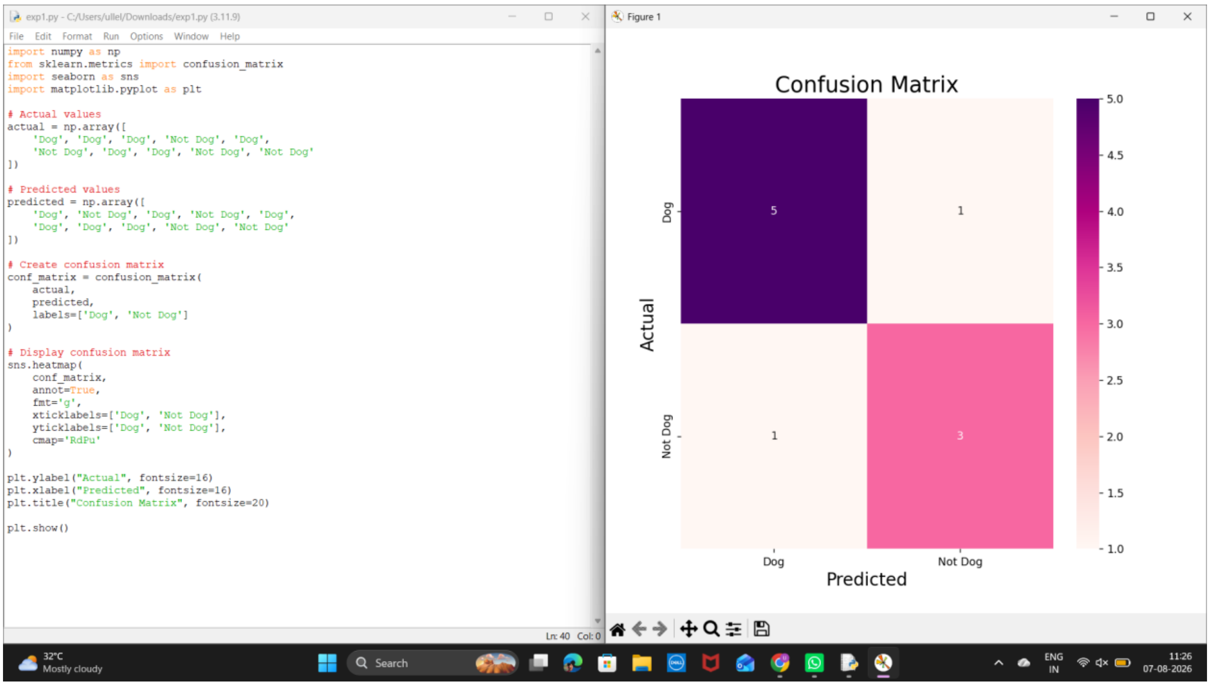
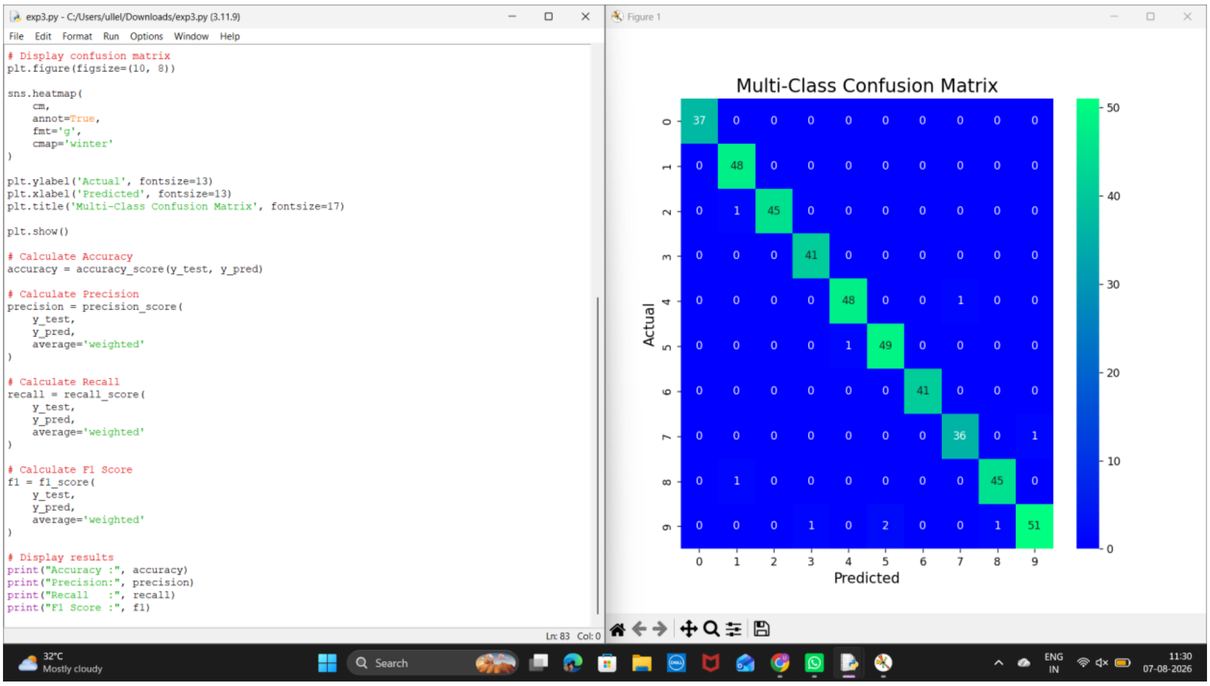
Lab Experiments

Experiment 1:

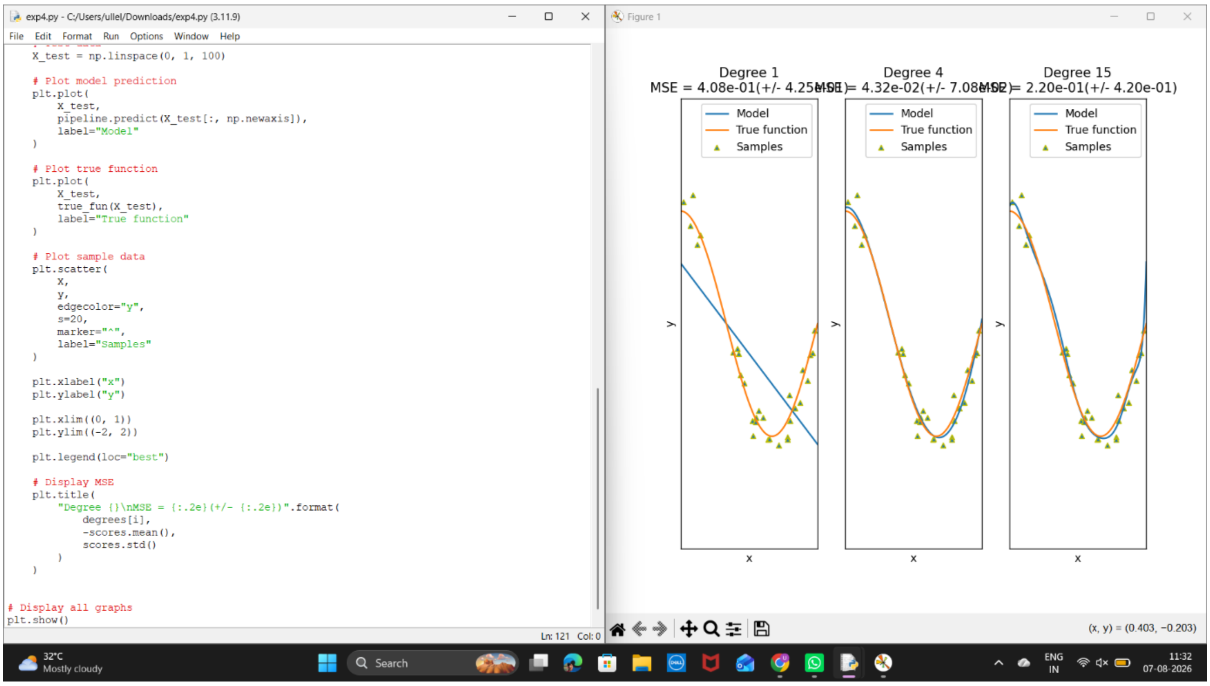


Experiment 2:

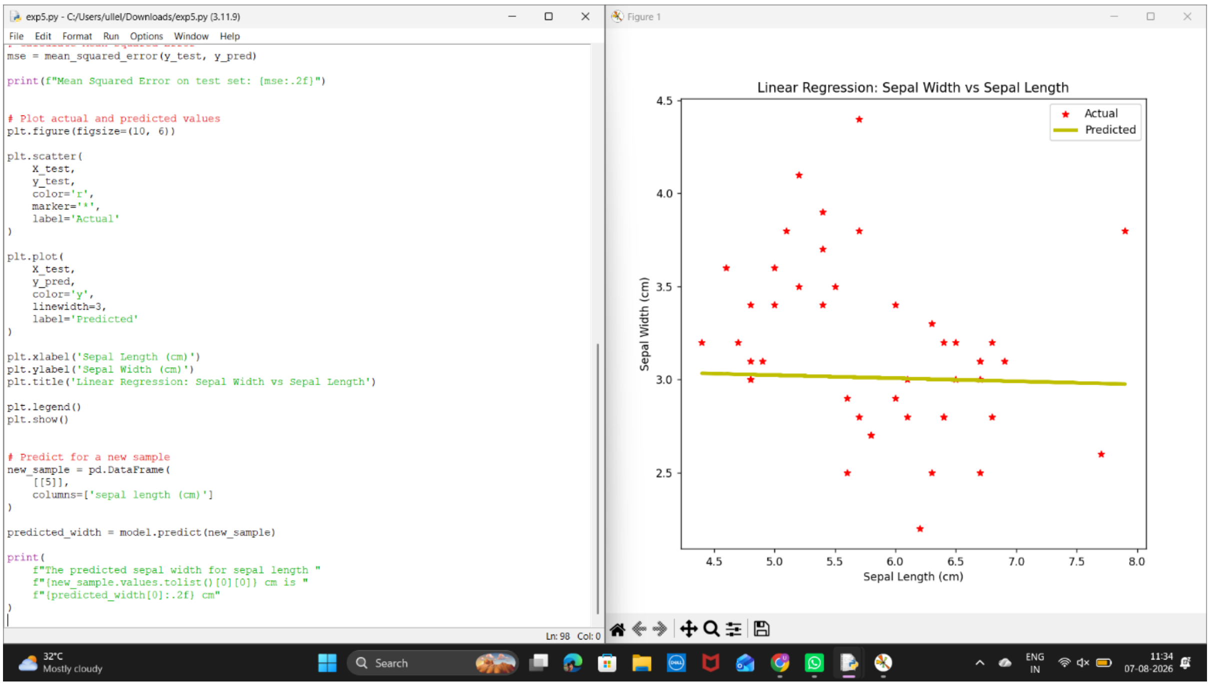
Experiment 3



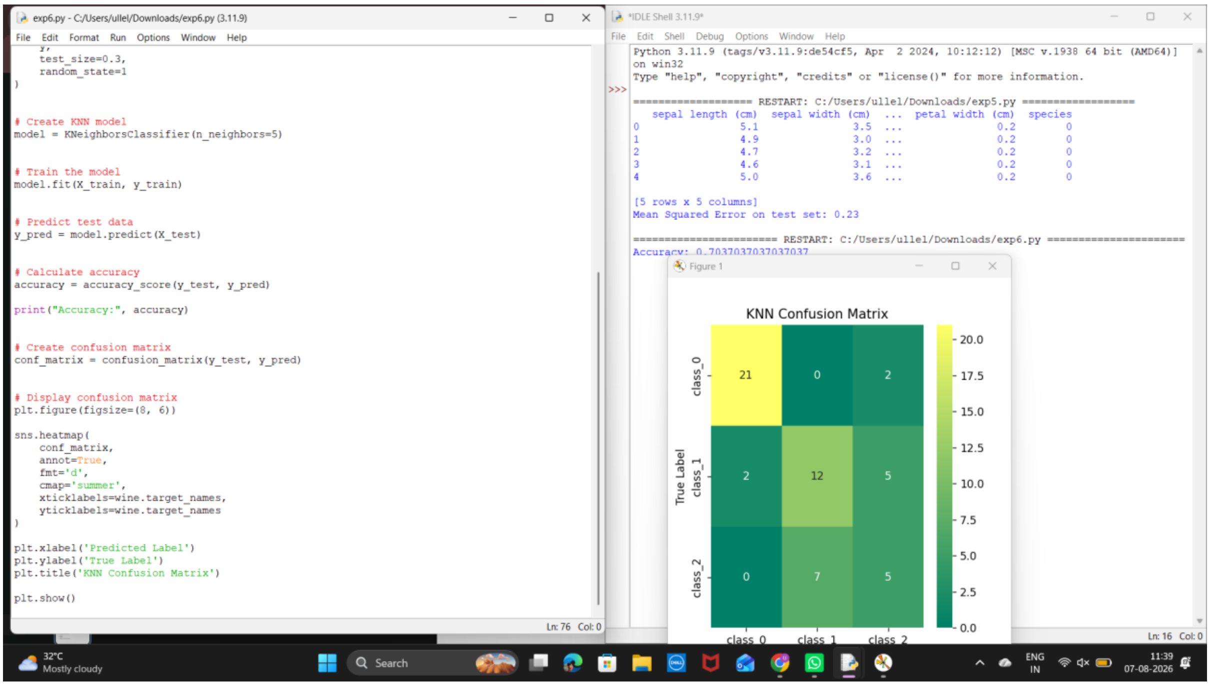
Experiment 4:



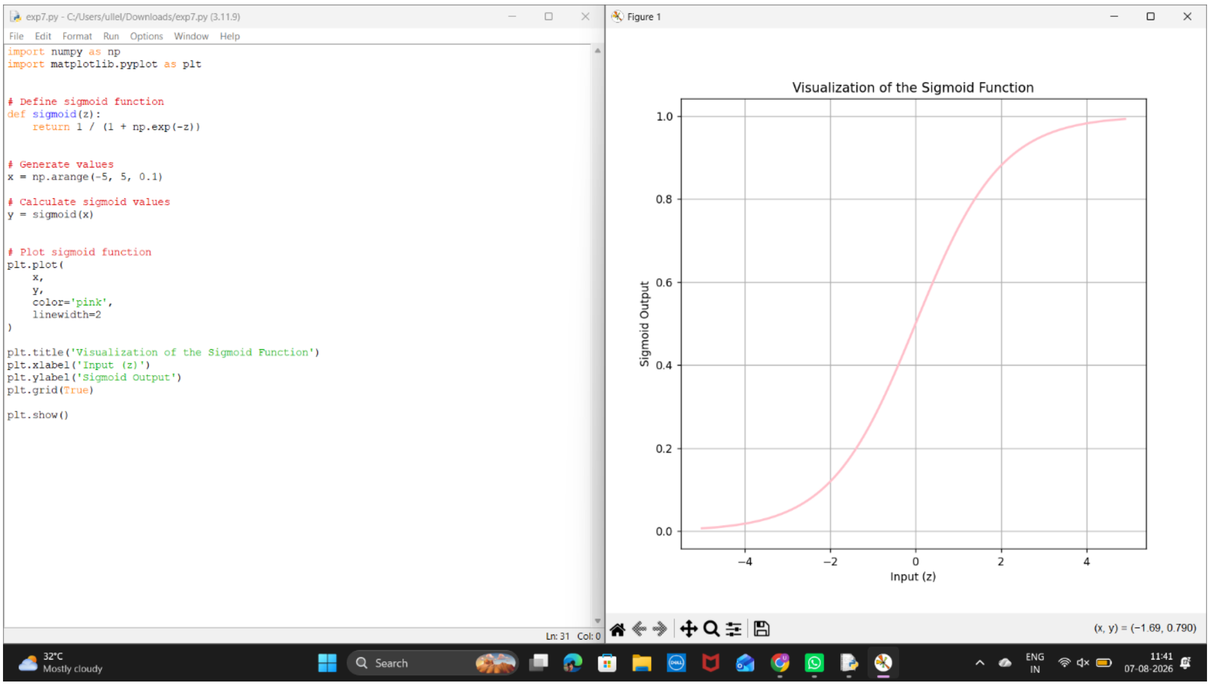
Exp5:



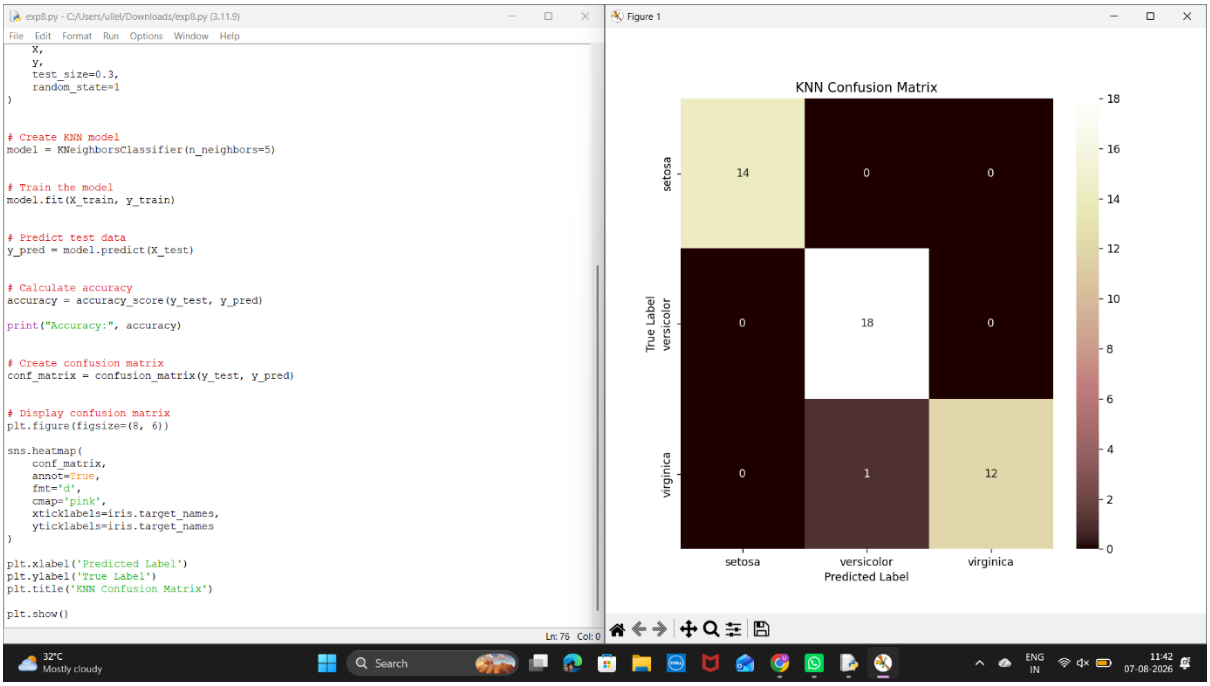
Experiment 6:



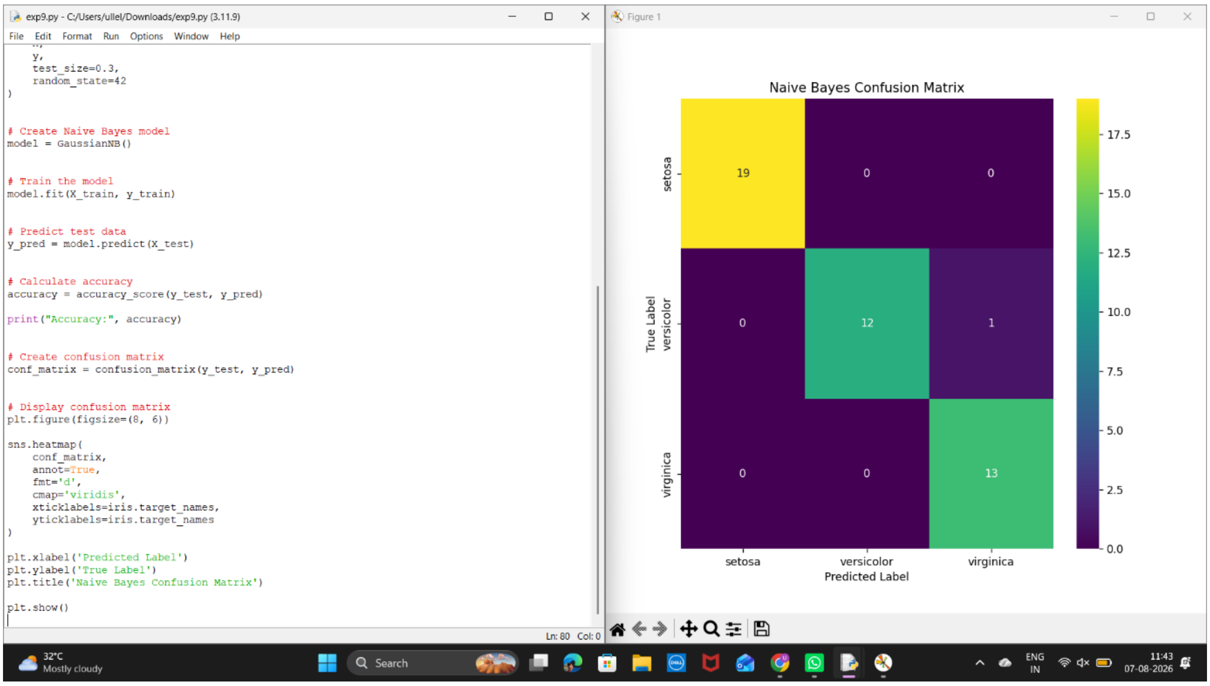
Exp 7:


exp7.py - C:/Users/ullel/Downloads/exp7.py (3.11.9)
File Edit Format Run Options Window Help
import numpy as np
import matplotlib.pyplot as plt
# Define sigmoid function
def sigmoid(z):
return 1 / (1 + np.exp(-z))
# Generate values
x = np.arange(-5, 5, 0.1)
# Calculate sigmoid values
y = sigmoid(x)
# Plot sigmoid function
plt.plot(
x,
y,
color='pink',
linewidth=2
)
plt.title('Visualization of the Sigmoid Function')
plt.xlabel('Input (z)')
plt.ylabel('Sigmoid Output')
plt.grid(True)
plt.show()
Ln: 31 Col: 0
Figure 1
Visualization of the Sigmoid Function
Sigmoid Output
Input (z)
(x, y) = (−1.69, 0.790)

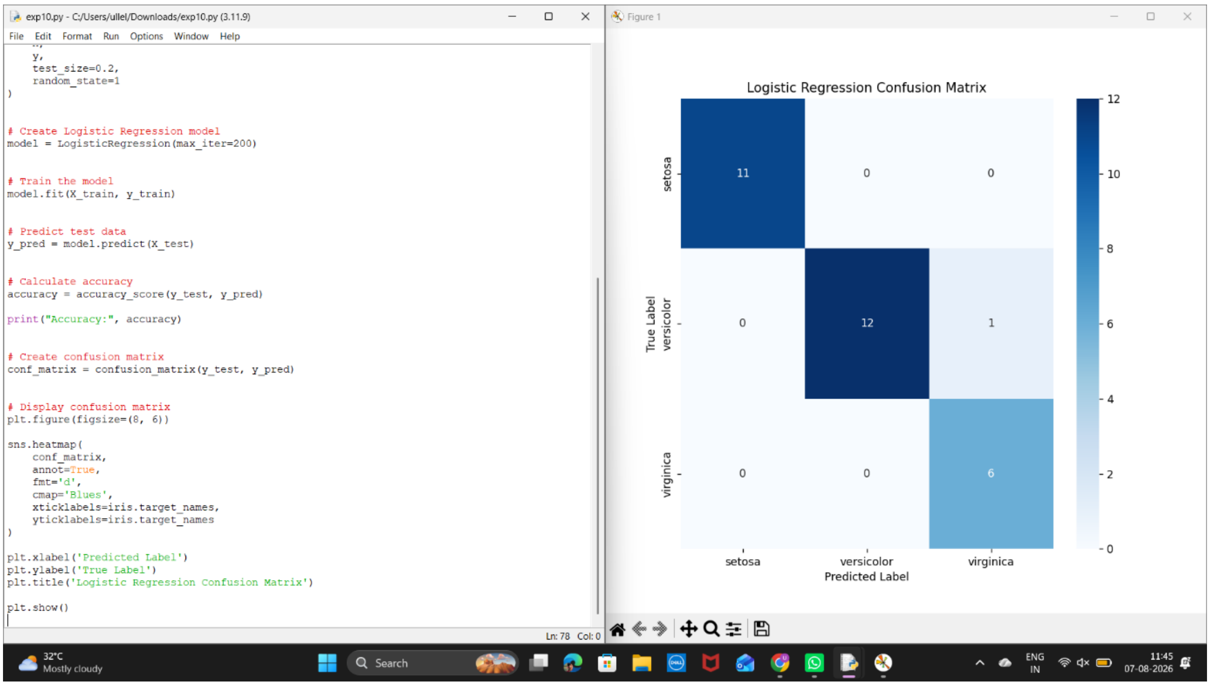
Exp 8:


exp8.py - C:/Users/ullel/Downloads/exp8.py (3.11.9)
File Edit Format Run Options Window Help
X,
y,
test_size=0.3,
random_state=1
)
# Create KNN model
model = KNeighborsClassifier(n_neighbors=5)
# Train the model
model.fit(X_train, y_train)
# Predict test data
y_pred = model.predict(X_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
# Create confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
# Display confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(
conf_matrix,
annot=True,
fmt='d',
cmap='pink',
xticklabels=iris.target_names,
yticklabels=iris.target_names
)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('KNN Confusion Matrix')
plt.show()
Ln: 76 Col: 0
Figure 1
KNN Confusion Matrix
setosa
versicolor
virginica
True Label
Predicted Label

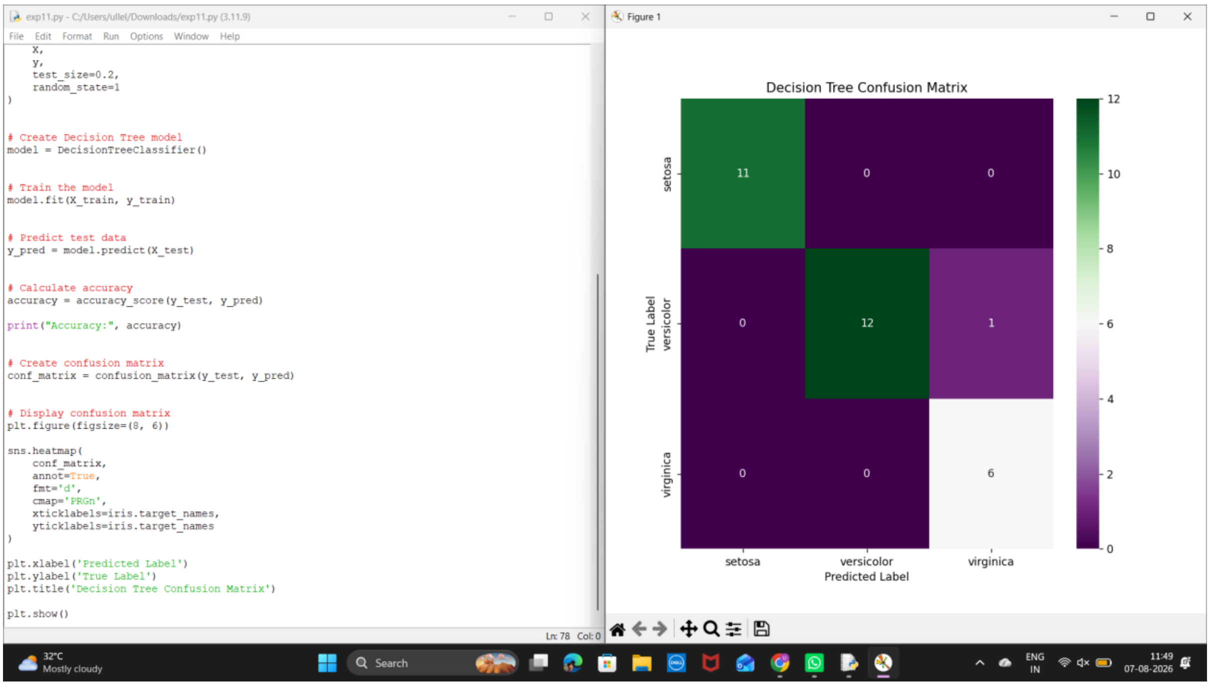
Exp 9:


exp9.py - C:/Users/ullel/Downloads/exp9.py (3.11.9)
File Edit Format Run Options Window Help
y,
test_size=0.3,
random_state=42
)
# Create Naive Bayes model
model = GaussianNB()
# Train the model
model.fit(X_train, y_train)
# Predict test data
y_pred = model.predict(X_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
# Create confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
# Display confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(
conf_matrix,
annot=True,
fmt='d',
cmap='viridis',
xticklabels=iris.target_names,
yticklabels=iris.target_names
)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Naive Bayes Confusion Matrix')
plt.show()
Ln: 80 Col: 0
Figure 1
Naive Bayes Confusion Matrix
setosa
versicolor
virginica
True Label
19 0 0
0 12 1
0 0 13
setosa
versicolor
virginica
Predicted Label
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
32°C
Mostly cloudy
Search
ENG
IN
11:43
07-08-2026

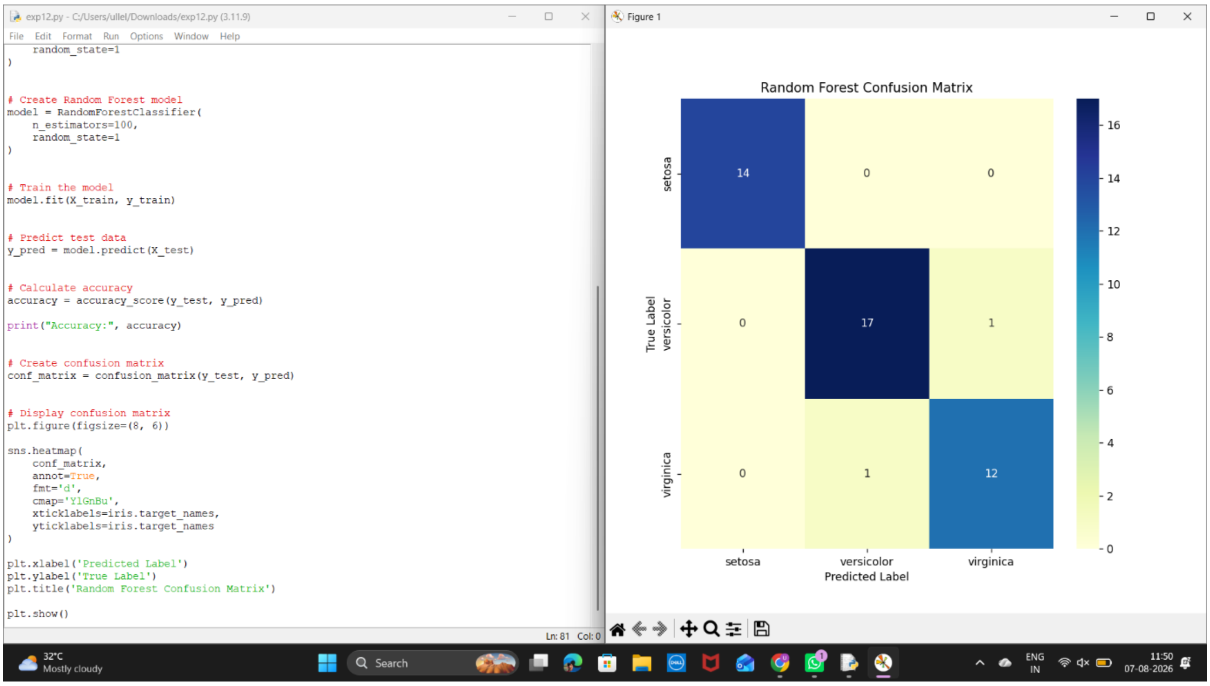
Exp10:


exp10.py - C:/Users/ullel/Downloads/exp10.py (3.11.9)
File Edit Format Run Options Window Help
y,
test_size=0.2,
random_state=1
)
# Create Logistic Regression model
model = LogisticRegression(max_iter=200)
# Train the model
model.fit(X_train, y_train)
# Predict test data
y_pred = model.predict(X_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
# Create confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
# Display confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(
conf_matrix,
annot=True,
fmt='d',
cmap='Blues',
xticklabels=iris.target_names,
yticklabels=iris.target_names
)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Logistic Regression Confusion Matrix')
plt.show()
Ln: 78 Col: 0
Figure 1
Logistic Regression Confusion Matrix
setosa
versicolor
virginica
True Label
11 0 0
0 12 1
0 0 6
setosa
versicolor
virginica
Predicted Label
12
10
8
6
4
2
0
32°C
Mostly cloudy
Search
ENG
IN
11:45
07-08-2026

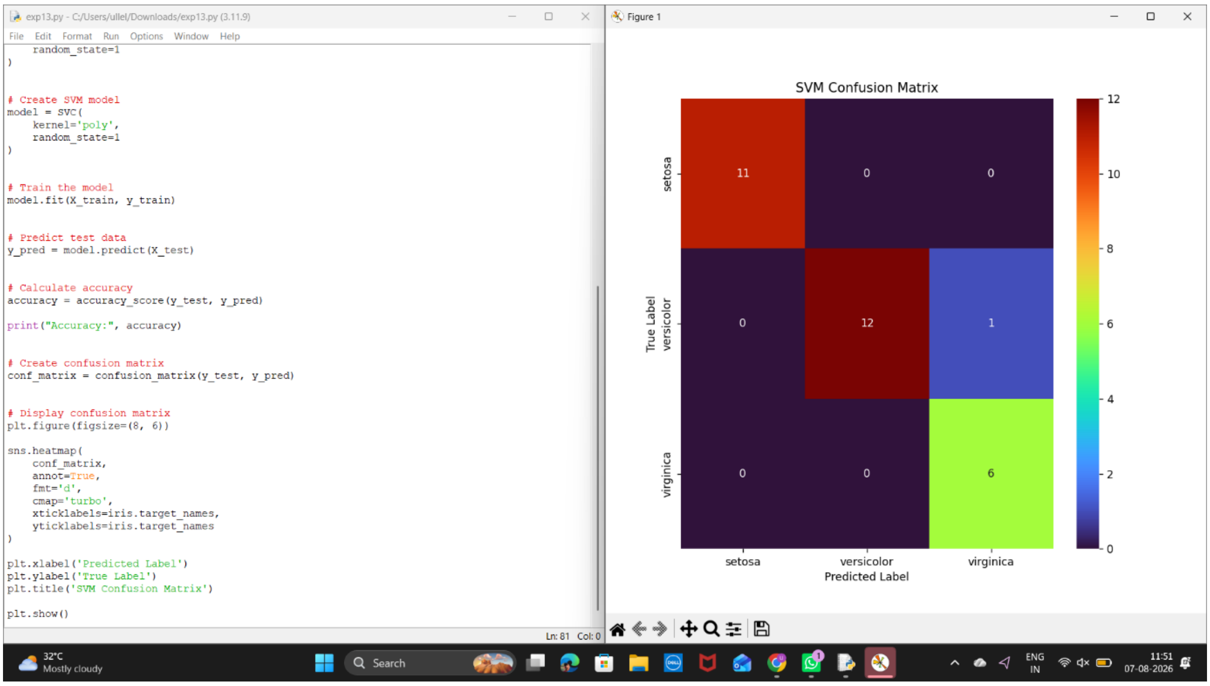
Exp11:


exp11.py - C:/Users/ullel/Downloads/exp11.py (3.11.9)
File Edit Format Run Options Window Help
X,
y,
test_size=0.2,
random_state=1
)
# Create Decision Tree model
model = DecisionTreeClassifier()
# Train the model
model.fit(X_train, y_train)
# Predict test data
y_pred = model.predict(X_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
# Create confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
# Display confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(
conf_matrix,
annot=True,
fmt='d',
cmap='PRGn',
xticklabels=iris.target_names,
yticklabels=iris.target_names
)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Decision Tree Confusion Matrix')
plt.show()
Ln: 78 Col: 0
Figure 1
Decision Tree Confusion Matrix
11
0
0
0
12
1
0
0
6
setosa
versicolor
virginica
True Label
Predicted Label
12
10
8
6
4
2
0
32°C
Mostly cloudy
Search
ENG
IN
11:49
07-08-2026

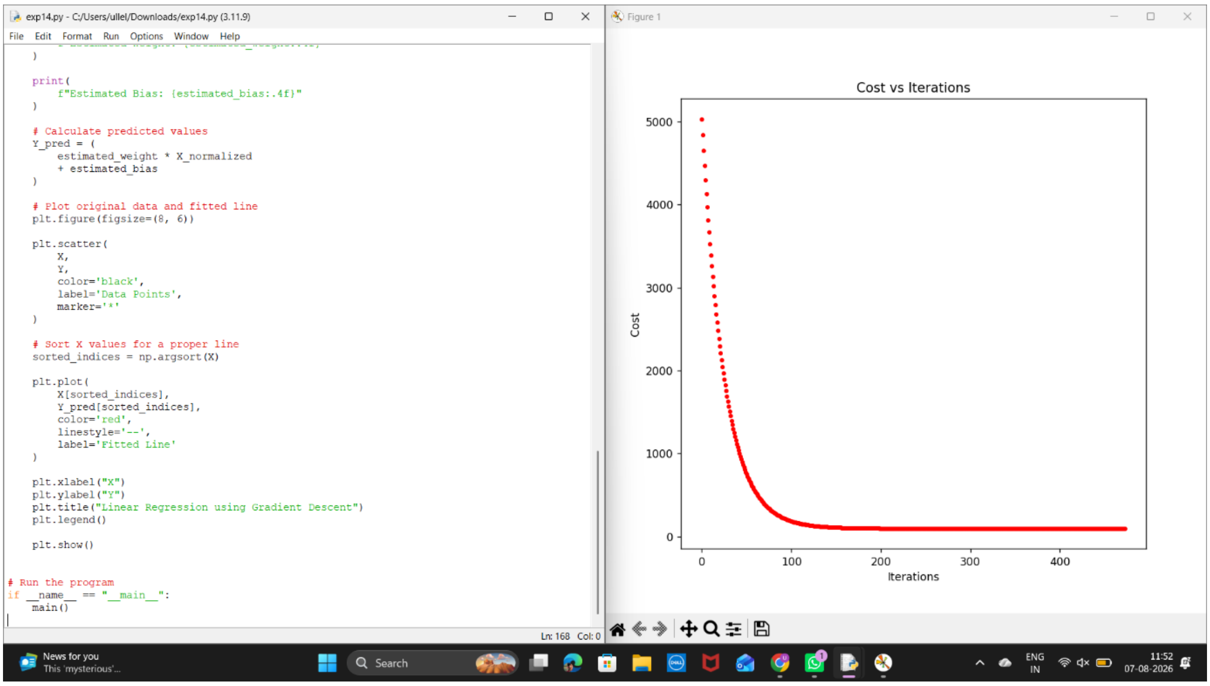
Exp 12:


exp12.py - C:/Users/ullel/Downloads/exp12.py (3.11.9)
File Edit Format Run Options Window Help
random_state=1
)
# Create Random Forest model
model = RandomForestClassifier(
n_estimators=100,
random_state=1
)
# Train the model
model.fit(X_train, y_train)
# Predict test data
y_pred = model.predict(X_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
# Create confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
# Display confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(
conf_matrix,
annot=True,
fmt='d',
cmap='YlGnBu',
xticklabels=iris.target_names,
yticklabels=iris.target_names
)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('Random Forest Confusion Matrix')
plt.show()
Ln: 81 Col: 0
Figure 1
Random Forest Confusion Matrix
14
0
0
0
17
1
0
1
12
setosa
versicolor
virginica
True Label
Predicted Label
16
14
12
10
8
6
4
2
0
32°C
Mostly cloudy
Search
ENG
IN
11:50
07-08-2026

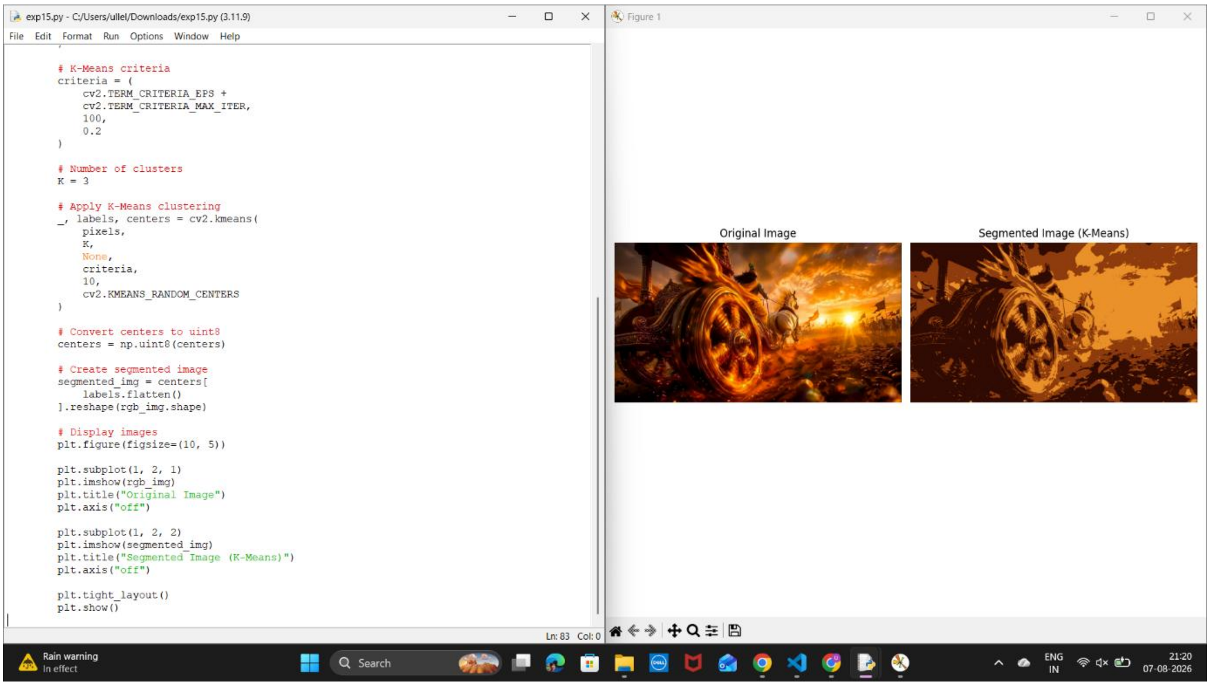
Exp 13:


exp13.py - C:/Users/ullel/Downloads/exp13.py (3.11.9)
File Edit Format Run Options Window Help
random_state=1
)
# Create SVM model
model = SVC(
kernel='poly',
random_state=1
)
# Train the model
model.fit(X_train, y_train)
# Predict test data
y_pred = model.predict(X_test)
# Calculate accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
# Create confusion matrix
conf_matrix = confusion_matrix(y_test, y_pred)
# Display confusion matrix
plt.figure(figsize=(8, 6))
sns.heatmap(
conf_matrix,
annot=True,
fmt='d',
cmap='turbo',
xticklabels=iris.target_names,
yticklabels=iris.target_names
)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.title('SVM Confusion Matrix')
plt.show()
Ln: 81 Col: 0
Figure 1
SVM Confusion Matrix
setosa
versicolor
virginica
True Label
11 0 0
0 12 1
0 0 6
setosa
versicolor
virginica
Predicted Label
12
10
8
6
4
2
0
32°C
Mostly cloudy
Search
ENG
IN
11:51
07-08-2026

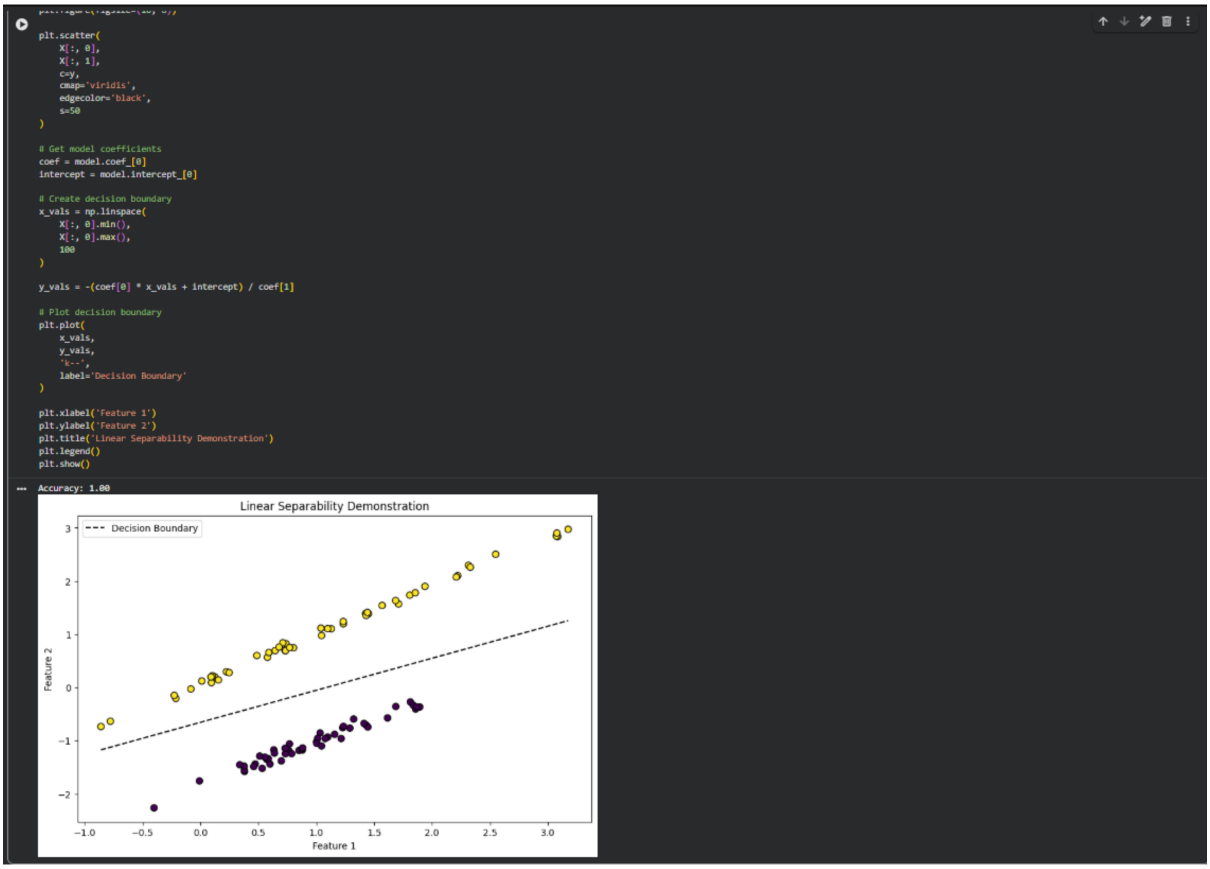
Exp 14:


exp14.py - C:/Users/ullel/Downloads/exp14.py (3.11.9)
File Edit Format Run Options Window Help
)
print(
f"Estimated Bias: {estimated_bias:.4f}"
)
# Calculate predicted values
Y_pred = (
estimated_weight * X_normalized
+ estimated_bias
)
# Plot original data and fitted line
plt.figure(figsize=(8, 6))
plt.scatter(
X,
Y,
color='black',
label='Data Points',
marker='*'
)
# Sort X values for a proper line
sorted_indices = np.argsort(X)
plt.plot(
X[sorted_indices],
Y_pred[sorted_indices],
color='red',
linestyle='--',
label='Fitted Line'
)
plt.xlabel("X")
plt.ylabel("Y")
plt.title("Linear Regression using Gradient Descent")
plt.legend()
plt.show()
# Run the program
if __name__ == "__main__":
main()
Ln: 168 Col: 0
Figure 1
Cost vs Iterations
Cost
5000
4000
3000
2000
1000
0
0
100
200
300
400
Iterations
News for you
Search
ENG
IN
11:52
07-08-2026

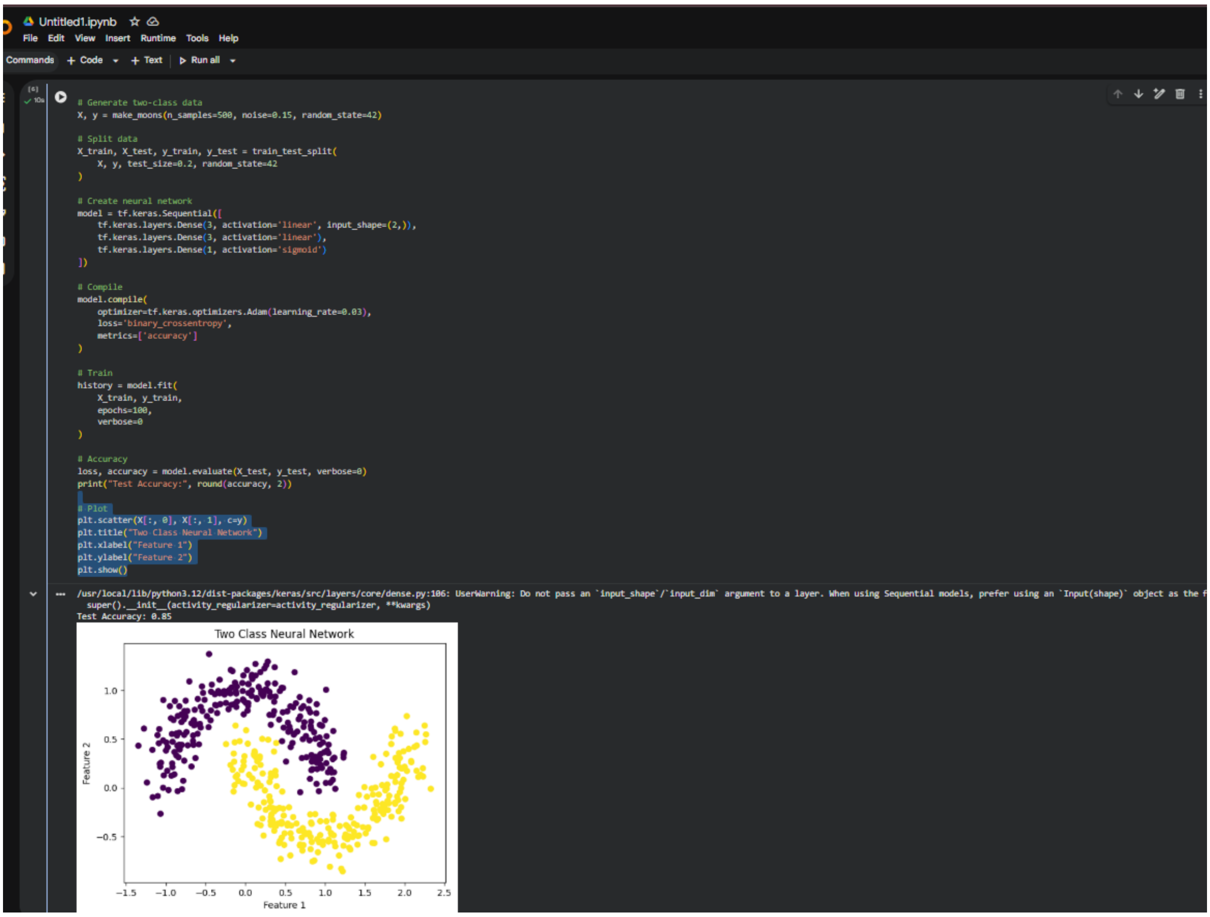
Exp 15:


exp15.py - C:/Users/ullel/Downloads/exp15.py (3.11.9)
File Edit Format Run Options Window Help
# K-Means criteria
criteria = (
cv2.TERM_CRITERIA_EPS +
cv2.TERM_CRITERIA_MAX_ITER,
100,
0.2
)
# Number of clusters
K = 3
# Apply K-Means clustering
_, labels, centers = cv2.kmeans(
pixels,
K,
None,
criteria,
10,
cv2.KMEANS_RANDOM_CENTERS
)
# Convert centers to uint8
centers = np.uint8(centers)
# Create segmented image
segmented_img = centers[
labels.flatten()
].reshape(rgb_img.shape)
# Display images
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(rgb_img)
plt.title("Original Image")
plt.axis("off")
plt.subplot(1, 2, 2)
plt.imshow(segmented_img)
plt.title("Segmented Image (K-Means)")
plt.axis("off")
plt.tight_layout()
plt.show()
Ln: 83 Col: 0
Figure 1
Original Image
Segmented Image (K-Means)
Rain warning
In effect
Search
ENG
IN
21:20
07-08-2026

Exp 17:


plt.scatter(
X[:, 0],
X[:, 1],
c=y,
cmap='viridis',
edgecolor='black',
s=50
)
# Get model coefficients
coef = model.coef_[0]
intercept = model.intercept_[0]
# Create decision boundary
x_vals = np.linspace(
X[:, 0].min(),
X[:, 0].max(),
100
)
y_vals = -(coef[0] * x_vals + intercept) / coef[1]
# Plot decision boundary
plt.plot(
x_vals,
y_vals,
'k--',
label='Decision Boundary'
)
plt.xlabel('Feature 1')
plt.ylabel('Feature 2')
plt.title('Linear Separability Demonstration')
plt.legend()
plt.show()
Accuracy: 1.00
Linear Separability Demonstration
Decision Boundary
Feature 1
Feature 2

Exp 18:


Untitled1.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text ▷ Run all

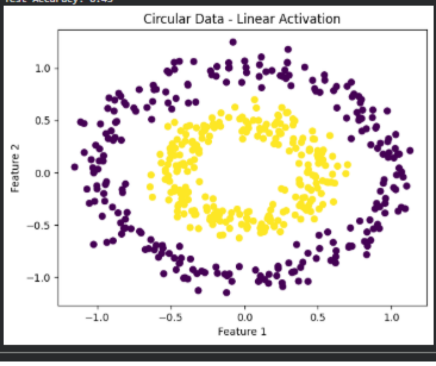
```python
# Generate two-class data
X, y = make_moons(n_samples=500, noise=0.15, random_state=42)

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create neural network
model = tf.keras.Sequential([
    tf.keras.layers.Dense(3, activation='linear', input_shape=(2,)),
    tf.keras.layers.Dense(3, activation='linear'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

# Compile
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.03),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

# Train
history = model.fit(
    X_train, y_train,
    epochs=100,
    verbose=0
)

# Accuracy
loss, accuracy = model.evaluate(X_test, y_test, verbose=0)
print("Test Accuracy:", round(accuracy, 2))

# Plot
plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Two Class Neural Network")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the f
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Test Accuracy: 0.85
```

Exp 19:

```python
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split

# Circular data
X, y = make_circles(
    n_samples=500,
    noise=0.08,
    factor=0.5,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Neural network
model = tf.keras.Sequential([
    tf.keras.layers.Dense(3, activation='linear', input_shape=(2,)),
    tf.keras.layers.Dense(3, activation='linear'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.03),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=100, verbose=0)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Accuracy:", round(accuracy, 2))

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Circular Data - Linear Activation")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

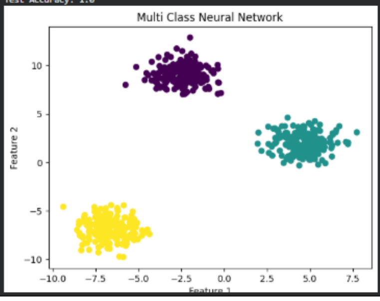
```
Test Accuracy: 0.43
```



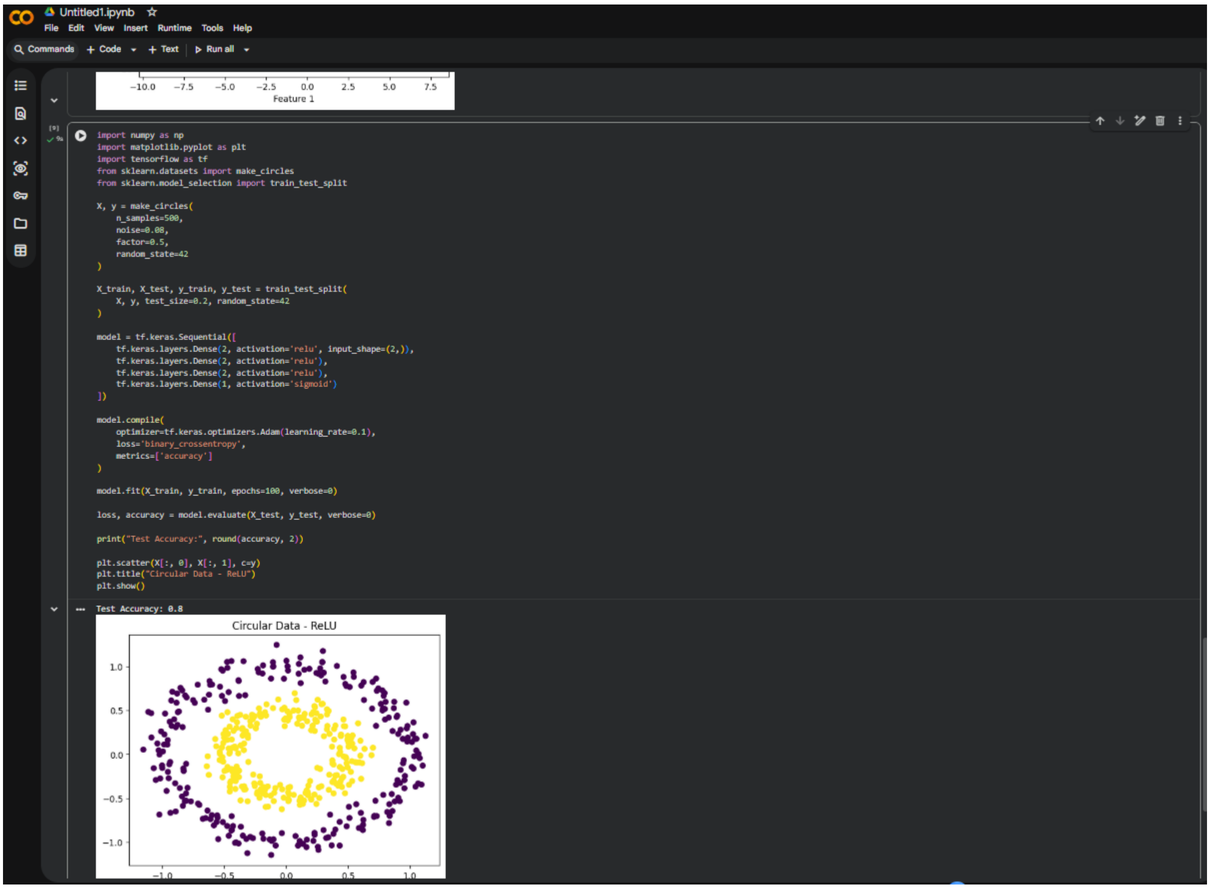
Exp 20:

```python
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split

# Generate multi-class data
X, y = make_blobs(
    n_samples=600,
    centers=3,
    n_features=2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Neural network
model = tf.keras.Sequential([
    tf.keras.layers.Dense(2, activation='linear', input_shape=(2,)),
    tf.keras.layers.Dense(2, activation='linear'),
    tf.keras.layers.Dense(3, activation='softmax')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.01),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=100, verbose=0)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Accuracy:", round(accuracy, 2))

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Multi Class Neural Network")
plt.xlabel("Feature 1")
plt.ylabel("Feature 2")
plt.show()
```

```
Test Accuracy: 1.0
```



Exp 21:



```python
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split

X, y = make_circles(
    n_samples=500,
    noise=0.08,
    factor=0.5,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(2, activation='relu', input_shape=(2,)),
    tf.keras.layers.Dense(2, activation='relu'),
    tf.keras.layers.Dense(2, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.1),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=100, verbose=0)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Accuracy:", round(accuracy, 2))

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Circular Data - ReLU")
plt.show()
```

```
Test Accuracy: 0.8
```

Exp 22:

```
[10]
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

X, y = make_moons(
    n_samples=500,
    noise=0.15,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(2, activation='relu', input_shape=(2,)),
    tf.keras.layers.Dense(2, activation='relu'),
    tf.keras.layers.Dense(2, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.1),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=100, verbose=0)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Accuracy:", round(accuracy, 2))

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Two Class Data - ReLU")
plt.show()
```

```
Test Accuracy: 0.43
```

Exp 23:

Untitled1.ipynb

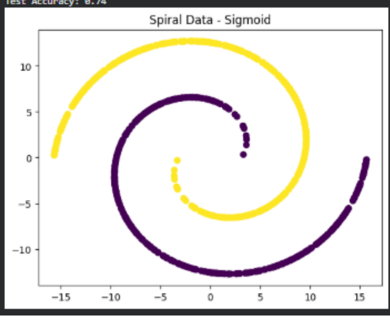
File Edit View Insert Runtime Tools Help

mmands + Code + Text Run all

[11]

```
x1 = np.column_stack([
    np.cos(theta) * r1,
    np.sin(theta) * r1
])

r2 = 2 * theta + np.pi
x2 = np.column_stack([
    np.cos(theta + np.pi) * r2,
    np.sin(theta + np.pi) * r2
])

X = np.vstack([x1, x2])
y = np.hstack([
    np.zeros(n),
    np.ones(n)
])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(3, activation='sigmoid', input_shape=(2,)),
    tf.keras.layers.Dense(3, activation='sigmoid'),
    tf.keras.layers.Dense(3, activation='sigmoid'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.1),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=200, verbose=0)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Accuracy:", round(accuracy, 2))

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Spiral Data - Sigmoid")
plt.show()
```

```
Test Accuracy: 0.74
```

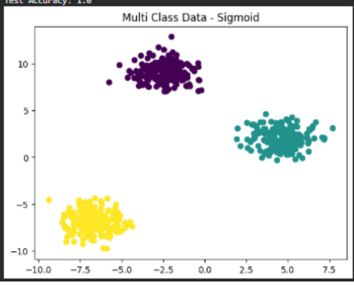


Exp 24:

[12]

```python
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split

X, y = make_blobs(
    n_samples=600,
    centers=3,
    n_features=2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(3, activation='sigmoid', input_shape=(2,)),
    tf.keras.layers.Dense(3, activation='sigmoid'),
    tf.keras.layers.Dense(3, activation='sigmoid'),
    tf.keras.layers.Dense(3, activation='softmax')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.1),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=100, verbose=0)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Accuracy:", round(accuracy, 2))

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Multi Class Data - Sigmoid")
plt.show()
```

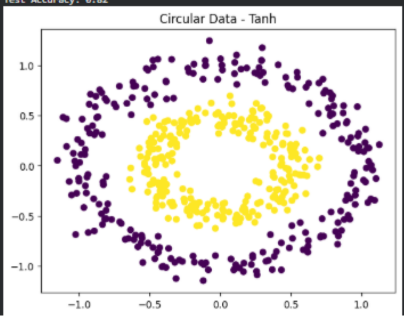
```
Test Accuracy: 1.0
```



Exp 25:

```
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_circles
from sklearn.model_selection import train_test_split

X, y = make_circles(
    n_samples=500,
    noise=0.08,
    factor=0.5,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(2, activation='tanh', input_shape=(2,)),
    tf.keras.layers.Dense(2, activation='tanh'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.1),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=100, verbose=0)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Accuracy:", round(accuracy, 2))

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Circular Data - Tanh")
plt.show()
```

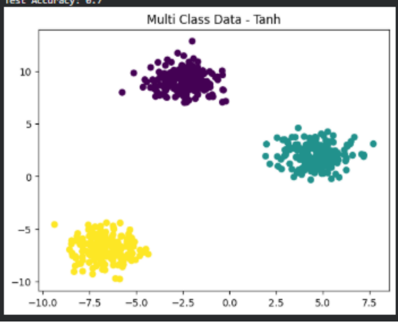
```
Test Accuracy: 0.82
```



Exp 26:

```python
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split

X, y = make_blobs(
    n_samples=600,
    centers=3,
    n_features=2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(2, activation='tanh', input_shape=(2,)),
    tf.keras.layers.Dense(2, activation='tanh'),
    tf.keras.layers.Dense(3, activation='softmax')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.1),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=100, verbose=0)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Accuracy:", round(accuracy, 2))

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Multi Class Data - Tanh")
plt.show()
```

```
Test Accuracy: 0.7
```



Exp 27:

colab.research.google.com/drive/18eXqLXaddV0tYPkp25kYrUF8Mj3KtetM

Untitled2.ipynb

```python
plt.title("Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.grid(True)

plt.show()
```

```
Final Results
-------------
Loss     : 0.6786280870437622
Accuracy : 0.625

Predictions
-----------
Input: [0. 0.] Actual: 0 Predicted: 0 Probability: 0.4665
Input: [0. 1.] Actual: 0 Predicted: 0 Probability: 0.4665
Input: [1. 0.] Actual: 0 Predicted: 0 Probability: 0.4665
Input: [1. 1.] Actual: 1 Predicted: 0 Probability: 0.4665
Input: [0.1 0.2] Actual: 0 Predicted: 0 Probability: 0.4665
Input: [0.2 0.1] Actual: 0 Predicted: 0 Probability: 0.4665
Input: [0.9 0.8] Actual: 1 Predicted: 0 Probability: 0.4665
Input: [0.8 0.9] Actual: 1 Predicted: 0 Probability: 0.4665
```

Exp 28:

```
[15]
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split

X, y = make_blobs(
    n_samples=600,
    centers=3,
    n_features=2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(2, activation='relu', input_shape=(2,)),
    tf.keras.layers.Dense(2, activation='relu'),
    tf.keras.layers.Dense(2, activation='relu'),
    tf.keras.layers.Dense(3, activation='softmax')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=100, verbose=0)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Accuracy:", round(accuracy, 2))

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Multi Class Data - ReLU")
plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Test Accuracy: 0.37
```

Exp29:

```
[16]
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_blobs
from sklearn.model_selection import train_test_split

X, y = make_blobs(
    n_samples=600,
    centers=3,
    n_features=2,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(3, activation='tanh', input_shape=(2,)),
    tf.keras.layers.Dense(3, activation='tanh'),
    tf.keras.layers.Dense(3, activation='tanh'),
    tf.keras.layers.Dense(3, activation='softmax')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.1),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=100, verbose=0)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Accuracy:", round(accuracy, 2))

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Multi Class Data - Tanh")
plt.show()
```

```
Test Accuracy: 1.0
```

Exp 30:

```python
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

X, y = make_moons(
    n_samples=500,
    noise=0.15,
    random_state=42
)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = tf.keras.Sequential([
    tf.keras.layers.Dense(3, activation='tanh', input_shape=(2,)),
    tf.keras.layers.Dense(3, activation='tanh'),
    tf.keras.layers.Dense(3, activation='tanh'),
    tf.keras.layers.Dense(1, activation='sigmoid')
])

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.1),
    loss='binary_crossentropy',
    metrics=['accuracy']
)

model.fit(X_train, y_train, epochs=100, verbose=0)

loss, accuracy = model.evaluate(X_test, y_test, verbose=0)

print("Test Accuracy:", round(accuracy, 2))

plt.scatter(X[:, 0], X[:, 1], c=y)
plt.title("Two Class Data - Tanh")
plt.show()
```

```
Test Accuracy: 1.0
```